

Assignment 4: Self-Supervised Representation Learning for Time Series Classification

Frederik Appel Vardinghus-Nielsen

September 27, 2026

Todo list

☐ Fill in.	14
☐ Fill in.	14
☐ Fill in.	15
☐ Fill in.	15
☐ Fill in.	15
☐ Fill in.	16
☐ Fill in.	16
☐ Fill in.	16
☐ Fill in.	17
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	18
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19
☐ Fill in.	19

Contents

1 Part A: Understand the methods and connect paper to code	4
1.1 The three learning ideas	4
1.1.1 T-Loss (Franceschi et al., NeurIPS 2019)	4

1.1.2	TS2Vec (Yue et al., AAAI 2022)	4
1.1.3	TF-C (Zhang et al., NeurIPS 2022)	5
1.2	Paper-to-code map	5
1.2.1	T-Loss	5
1.2.2	TS2Vec	7
1.2.3	TF-C	10
1.2.4	Following one TF-C batch through the pipeline	12
2	Part B: Raw-signal baselines	14
2.1	Protocol	14
2.2	Results	14
2.3	Comments	14
3	Part C: Pretrain and evaluate T-Loss and TS2Vec	15
3.1	Protocol	15
3.2	Results	15
3.3	Comparison with the raw-signal baselines	15
3.4	Training time and representation dimensionality	16
4	Part D: Robustness to contiguous sensor dropout	16
4.1	Corruption design	16
4.2	Results (100%-label models)	16
4.3	Discussion	16
5	Part E: Focused TS2Vec continuous masking ablation	16
5.1	Setup	16
5.2	Results	17
5.3	Interpretation	17
6	Part F: TF-C for industrial bearing fault classification (FD-A $\rightarrow$ FD-B)	17
6.1	Execution record	17
6.2	Classification result	17
6.3	What was transferred, where labels entered, and encoder status	18
6.4	Material differences between the paper’s protocol and the supplied execution path	18
7	Part G: Synthesis	18
7.1	When SSL provides a useful representation and when raw nonlinear classification remains competitive	18
7.2	Whether the effect of SSL becomes clearer with only 10% labels	18
7.3	Stability of conclusions across the paired seeds	18
7.4	Role of dataset size, dimensionality and number of classes	18
7.5	Whether continuous masking improves accuracy, robustness, both or neither	18
7.6	What the TF-C bearing experiment shows about transfer between operating conditions	18
7.7	Implications for future multichannel industrial sensor data	18
8	Experimental rules: compliance notes	19
9	Limitations and conclusions	19
A	Optional extension: TF-C on FordA	19
B	Optional extension: FordB noisy-condition experiments	19

1 Part A: Understand the methods and connect paper to code

1.1 The three learning ideas

1.1.1 T-Loss (Franceschi et al., NeurIPS 2019)

The general idea is that sub-series of a reference series (which may be a sub-series of a larger time series) is a positive sample, which sub-series of another series are negative samples. They use a CNN with dilation for increased receptive field. The series are left padded with zero if they are too short for the dilation of the network. If the series are long, more time stamps have representations in the end which is fixed with a max pool.

- Positive/negative samples: a positive example x^{pos} is a sub-series of the reference series x^{ref} , which itself is randomly sampled in length and location. A sub-series of another time series is negative.
- Objective: triple loss with one anchor, one positive, and $K \geq 1$ negative samples x_k^{neg} . The objective is to have the distance between x^{pos} and x^{ref} be smaller than the sum of the distances between x^{ref} and x_k^{neg} .
- Fixed length problem: the convolutional network ends with a representation $Y \in \mathbb{R}^{160 \times L}$ which is max pooled into $z \in \mathbb{R}^{160}$ and thereafter transformed through $f(x) = Wz + b$ where $W \in \mathbb{R}^{320 \times 160}$. The final dot product are therefore always between vectors of length 320. If the time series chosen in the beginning are too short for the full CNN they are left zero padded.
- Classification: an SVM with a radial basis function is trained on the features with their labels now available. It is evaluated in a train/test split.

1.1.2 TS2Vec (Yue et al., AAI 2022)

The general idea is to generate related views with masking and random overlapping segments of the representations. The hierarchical loss furthermore lets the encoder learn representations which take into account a spectrum of resolutions (granularity of the series).

- Positive/negative samples: two different views of the same time series instance (segment) are achieved through random cropping and timestamp masking. The original time series is cropped to produced two distinct, overlapping time series. After an input projection layer, these are both randomly masked at specific time stamps (single timestamp, all dimensions for multivariate TS) by setting values to zero.
- Objective: Dual loss, the sum of two distinct expressions:
 - Temporal contrastive loss which compares views of the same overlapping series at the same and different timestamps. Similarity at same timestamp between two different views is positive, self-similarity (same view, no masking) at different timestamps is negative.
 - Instance-wise contrastive loss compares to other time series. Positives are two views of the same time series at the same timestamp. Negatives are both view from another time series at the same time stamp.

The loss is calculated recursively at larger and larger granularity with a dilation of 2. The first level compares every single timestamp with all others. The second max pools with kernel size 2 and this continues until the full series are used.

- Fixed length problem: only the overlapping timestamps are used after passing the views through the encoder. The loss loops over time stamps and gives a single loss for them all.
- Classification: they use instance-level representations (the whole segment of time series, not just the overlap). Instances are passed through the encoder and a max pooling is performed over all timestamps, giving a feature vector of dimension K , which is the number of representation dimensions in the last layer. SVM is trained on this K -dimensional vector for classification.

1.1.3 TF-C (Zhang et al., NeurIPS 2022)

They seek to enforce discrimination in different domains This encourages that the time domain is robust to augmentation, the frequency domain is robust to augmentation, and the time-frequency domain enforces temporal and spectral consistency.

- Positive/negative samples:
 - Time: (1) Encodings of a time series and its augmented (jittering, scaling, time-shifts, neighborhood segments) view are similar and (2) encodings of two different time series (no augmentations) are dissimilar.
 - Frequency: (1) Encodings of the Fourier transform of a time series and its augmented view are similar and (2) encodings of two different series (no augmentations) are dissimilar.
 - Time-frequency: (1) Time series and their Fourier transform must be similar after first encoding into respective domains and thereafter encoding into common time-frequency domain, (2), augmented views are dissimilar.
- Objective: Combined loss in time-, frequency-, and time-frequency domain.
- Fixed length problem: The training datasets all contain series of same length. When using the model on a tuning dataset and testing, the series are left zero padded.
- Classification: the full architecture of $z_i^T = R_T(G_T(x_i^T))$ and $z_i^F = R_F(G_F(x_i^F))$ is used and concatenated into $[z_i^T, z_i^F]$ and this is passed through a 2-layer fully ocnnected network which projects into the number of classes needed. This classifier is jointly trained with the full pipeline on the target dataset as fine tuning.

1.2 Paper-to-code map

For each element: (i) closest paper location, (ii) inputs and outputs, (iii) immediate caller or call site, and (iv) role in training or representation extraction. Verified checkout commits: T-Loss 4aff592, TS2Vec b0088e1, TF-C 9667582.

1.2.1 T-Loss

`losses/triplet_loss.py: TripletLoss.forward`

- (i) Eq. (1)
- (ii) **Input:** tensor (B, C, L), where B is the batch size, C is the number of channels, and L is the length of the time series (equal length case).
Output: Loss as a scalar.
- (iii) `TimeSeriesEncoderClassifier.fit_encoder`
- (iv) It is the loss which dictates which time series are supposed to have similar and dissimilar representations.

`networks/causal_cnn.py: CausalConvolutionBlock.forward`

- (i) Figure 2(b) in the paper.
- (ii) **Input:** tensor (B, C, L), where B is the batch size, C is the number of channels, and L is the length of the time series (equal length case).
Output: tensor (B, C, L), where B is the batch size, C is the number of channels, and L is the length of the time series (equal length case). This is the results after passing through the convolutional block.
- (iii) CausalCNN where is is called for each layer in the full network.
- (iv) It's the basic block which dictates the neural network achitecture and therefore the inductive bias of the representation learning.

`networks/causal_cnn.py: CausalCNNEncoder.forward`

- (i) Section 4 describes the full architecture. Figure 2 illustrates the convolutional blocks and dilation which are repeated in the first part of the encoder.
- (ii) **Input:** tensor (B, C, L), where B is the batch size, C is the number of channels, and L is the length of the time series (equal length case).
Output: tensor (B, C), where B is the batch size and C is the number of channels. The time dimension L is removed with max pooling squeezing the time dimension.
- (iii) `TimeSeriesEncoderClassifier.encode`
- (iv) It wraps the whole classification pipeline of the encoder and calls it on the passed batch.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.fit_encoder`

- (i) Section 4 on architecture and Section S2 on hyperparameters.
- (ii) **Input:** Training set X .
Output: The fitted encoder.
- (iii) `TimeSeriesEncoderClassifier.fit`
- (iv) It performs unsupervised/self-supervised fitting of the encoder.

`scikit_wrappers.py: TimeSeriesEncoderClassifier.encode`

- (i) Paragraph 4 and 5 in Section 4 on encoder architecture.
- (ii) **Input:** The test set X .
Output: Features of X .
- (iii) Depends on the context. In general `fit()` calls it during training to sue the final encoder to produce features before training SVM, while `score()` calls is during testing to produce features for SVM classification.
 - `fit_encoder()` calls it after each training epoch if certain conditions for early stoping are present. This doesn't make sense for fully self-supervised learning, so labels must be provided.
 - `fit()` calls it to encode the training set once training is done such that SVM can be training on the training features.

- `encode_window()` calls it to encode a test set on a sliding window. Useful if temporal features are needed for regression/forecasting.
 - `predict()` and `score()` call it to encode a set and then perform prediction or calculate score of classification. Mostly used for testing.
- (iv) It uses the final encoder (unless used during early stopping) to encode features from either train or test sets before classification or prediction.

Required specifics

- How `fit_encoder` selects the fixed-length or varying-length loss: It checks whether a `numpy` sum over the training set yields `nans` in some entries, which then indicates unequal lengths.
- How `TripletLoss.forward` samples reference, positive and negative intervals:
 - Reference/anchor: each time series is used as reference once per epoch. The length is chosen at random between the length of the positive example and the length of the full series. The position is chosen to overlap the positive example.
 - Negative: a specified number of random time series are chosen as negatives for each anchor. The anchor itself can contain a negative sample.
 - Positive: chosen as a random sub-series of each anchor. In practice the length and position of the positive is chosen first.
- How `CausalCNNEncoder` produces one fixed-length vector: it uses `AdaptiveMaxPool1d` to max pool over the time dimension. This produces a feature vector which only has length one in the remaining "time" dimension.
- How to call `fit_encoder` without labels: call it without passing training labels `y`. It defaults to none and performs unsupervised learning without early stopping.

1.2.2 TS2Vec

`ts2vec.py: TS2Vec.fit`

- (i) Section 2.3 on Contextual Consistency which describes the timestamp masking and random cropping which serve as the augmentations for robustness, and Section 2.4 on Hierarchical Contrasting which is described in Algorithm 1. The whole loss is shown in Equations (1), (2), and (3).
- (ii) **Input:** Train data, number of epochs to train, maximum number of iterations (batches) to train, and whether to print training loss after each epoch.
Output: Log of losses during training.
- (iii) It's called in `train.py`.
- (iv) It contains the central training loop which includes cropping, augmentation, and loss.

`ts2vec.py: TS2Vec._eval_with_pooling`

- (i) Sections 3.1, 3.2, and 3.3 on applications. The evaluation/representation of the time series varies depending on the downstream task.
- (ii) **Input:** Test data `x` on which to evaluate the averaged model, whether to use a mask, whether to slice.
Output: Representations from running the encoder and the chosen pooling strategy.

- (iii) `TS2Vec.fit()`
- (iv) It applies the trained model to a test set and the pooling strategy which is chosen based upon the downstream task.

`ts2vec.py: TS2Vec.encode`

- (i) Section 2.2 on Model Architecture and Sections 3.1, 3.2, and 3.3 on downstream tasks.
- (ii) **Input:** Test data to calculate representations from. Masking, sliding length and padding for representation altering, encoding window for restriction before encoding, causal, and batch sizes are additional inputs.
Output: The representations from encoding the test data.
- (iii) `anomaly_detection.py`, `classification.py`, and `forecasting.py` for the three tasks, respectively.
- (iv) Uses the fitted encoder to encode test data for use in downstream task.

`models/losses.py: hierarchical_contrastive_loss`

- (i) Equations (1), (2), and (3).
- (ii) **Input:** Two different representation of the same batch to calculate loss between.
Output: Loss between the representations as defined by Equation (3).
- (iii) `TS2Vec.fit()`
- (iv) Calculates loss which dictates training.

`models/losses.py: instance_contrastive_loss`

- (i) Equation (2).
- (ii) **Input:** Two representations of the same batch after encoding during training.
Output: The instance contrastive loss between the two representations. Two representations at the same timestamp are positive, while same representation of two different time series are negative.
- (iii) `hierarchical_contrastive_loss`.
- (iv) Forces the model to have representations of the same series in different contexts and with different masks to be similar, while keeping representations of other series dissimilar.

`models/losses.py: temporal_contrastive_loss`

- (i) Equation (2).
- (ii) **Input:** Two representations of the same batch after encoding during training.
Output: The temporal contrastive loss between the two representations. The same timestamp for the two representations is a positive pair, while different timestamps in both the same and the other representation are negative pairs.
- (iii) `hierarchical_contrastive_loss`.
- (iv) Encourages the model to discriminate representations over time. Representations at the same timestamps should have similar representations while temporal differences should be reflected in different representations.

`models/encoder.py: TSEncoder.forward`

- (i) Section 2.2, paragraphs 2 and 3.
- (ii) **Input:** Data to be encoded. It has been cropped and augmented before.
Output: Encoded batch.
- (iii) `TS2Vec.fit()` and `_eval_with_pooling`.
- (iv) Contains the convolutional part of the model architecture.

`models/encoder.py: generate_binomial_mask`

- (i) The paragraph **Timestamp Masking** in Section 2.3.
- (ii) **Input:** B, T, and p; batch size, time length, and masking probability, respectively.
Output: A boolean tensor with shape (B, T) with values **False** and **True** indicating masking or not, respectively.
- (iii) `TSEncoder.forward()`
- (iv) Performs the timestamp masking which forces robustness and context understanding of the model.

`models/encoder.py: generate_continuous_mask`

- (i) Not mentioned in article.
- (ii) **Input:** B, T, n, and l; batch size, time length, number of masked sequences, and length of masked sequences as a fraction of full length, respectively.
Output: A boolean tensor with shape (B, T) with values **False** and **True** indicating masking or not, respectively. The **while** loop in `hierarchical_contrastive_loss` matches the loop in Algorithm 1.
- (iii) `TSEncoder.forward()`
- (iv) Creates continuous runs of masking. They may overlap.

Required specifics

- Overlapping crops in `TS2Vec.fit`: They are created by in sequence determining the length of the overlap of the crops, the start and end of the overlap, the extension of crops 1 to the left, and finally the extension of crop 2 to the right.
- Where timestamp masking is applied: In `TSEncoder.forward()`. This is only applied during training.
- Mapping of the two elementary losses and hierarchical pooling to Equations 1–3 and Algorithm 1: `temporal_contrastive_loss`, `instance_contrastive_loss`, and `hierarchical_contrastive_loss` match Equations (1)–(3), respectively.
- The `TS2Vec.encode` option returning one vector per complete series: Set the argument `encoding_window="full_series"`. This causes a max pool of the features over all time stamps after encoding.

1.2.3 TF-C

code/TFC/dataloader.py: `LoadDataset.__init__`

- (i) Sections 4.1 and 4.2 on data augmentation in time and frequency domains.
- (ii) **Input:** A dataset with time series and labels, a config, training mode, max size of a debugging dataset, and whether to debug (use a smaller dataset).
Output: No output. Saves two version of the time series: the augmented version and the Fourier transformed and then augmented.
- (iii) Three times in `data_generator`; once for each of training, fine-tuning, and testing.
- (iv) Performs the steps of loading and augmenting data before it is passed through the first set of encoders.

code/TFC/dataloader.py: `LoadDataset.__getitem__`

- (i) Has no direct counterpart in the article. It just serves to pop out a single time series, its augmentations, and transforms.
- (ii) **Input:** Index of the requested time series.
Output: The requested time series, its augmentation in time, the Fourier transform, its augmentation in frequency, and the labels. if run at fine tuning or test time, the augmentations are the same as the original representations.
- (iii) `model_pretrain`, `mode_finetune`, and `mode_test`.
- (iv) Is a wrapper for providing the augmented and transformed views and labels during training, fine tuning, and test.

code/TFC/dataloader.py: `data_generator`

- (i) No direct counterpart. Paragraph 4 of Section 1 explains the concept of pre-training, fine-tuning, and then testing. This split is done by this generator.
- (ii) **Input:** Path to the source data, path to the target (fine tuning and test dataset), configs, training mode (pre-train, fine tuning, or test), and whether to run in debugging mode.
Output: Loaders for the three different cases and datasets: training, fine tuning, and testing.
- (iii) `main.py`
- (iv) Helps loading the correct datasets with the corresponding augmentations for pre-training, fine tuning, and testing.

code/TFC/dataloader.py: the `fft.fft(...).abs()` operation

- (i) Section 4.2, paragraph 1, where the frequency representation is described as the Fourier transformation of the time series.
- (ii) **Input:** The whole dataset of time series.
Output: The absolute values of the Fourier transform of the time series.
- (iii) It is saved in the class instance as the spectral representation of the time series. During training, it is called in `TFC.forward()`.
- (iv) Is responsible for the spectral representation which the authors deem valuable for time series representation.

code/TFC/augmentations.py: DataTransform_TD, DataTransform_FD, remove_frequency, add_frequency

- (i) Sectio 4.2, paragraph 2 on augmentations of the Fourier transform of the time series.
- (ii) **Input:** Fourier transform of a single time series.
Output: An augmented Fourier transform where frequencies are added and removed at random.
- (iii) Load_Dataset.__init__() when augmenting the Fourier transformed time series.
- (iv) Performs augmentations of the absolute Fourier transform of the raw time series in order to provide positive pairs in the spectral representations. The code is wrongly implemented and does not actually remove frequencies but rather in practice amplifies others.

code/TFC/model.py: TFC.__init__, TFC.forward, target_classifier.forward

- (i) Calls and structures the overall architecture and data flow in the model. `init` defines the four encoders, `forward` defines the four representations when the time series, the Fourier transforms, and their augmentments are passed through the encoders, and `target_classifier.forward` is simply a small, fully connected neural network which is training during fine tuning and used for classification during testing.
- (ii) **Input:** Training data and its Fourier transform.
Output: The four representations encoded by the various combinations of the encoders.
- (iii) `main.py`
- (iv) This structures all of the defined architectures into the whole model and defines the flow of data through the various networks and classifiers.

code/TFC/loss.py: NTXentLoss_poly.forward

- (i) Equations (1), (2), and (3).
- (ii) **Input:** Representations Z and $\tilde{Z}$.
Output: Total loss from postive and negative samples in the two representations.
- (iii) It is called in `trainer.py`: `model_pretrain` and `model_finetune` for use during training and fine tuning.
- (iv) Implements the well known NT-Xent loss between the various positive and negative samples in the model.

code/TFC/trainer.py: Trainer, model_pretrain, model_finetune, model_test

- (i) Sections 5.1 and 5.2 on results from pre-training and fine-tuning.
- (ii) **Input:** Model, optimizer, classifier, classifier optimizer, dataloaders, and configuration files.
Output: No returns. Prints logging during training or fine tuning and saves models and results.
- (iii) `main.py`
- (iv) `Trainer` splits the model in pre-training and fine-tuning and runs the overall logic for pre-training and fine-tuning and saves the models, `model_pretrain` runs the pre training pipeline and returns the loss, and `model_finetune` does the same for the test datasets and reports statistics on the results.

code/TFC/main.py: the subset setting, call to `data_generator`, pretrained-checkpoint loading

- (i) No directly corresponding parts of the article.
- (ii) **Input:** Not a function, so no inputs. However, needs configuration files and datasets.
Output: No outputs but logs progress during pre-training of fine-tuning. The models which are trained are saved to the local machine during this function.
- (iii) No immediate caller.
- (iv) Top level file for entry for both pre-training and fine-tuning.

1.2.4 Following one TF-C batch through the pipeline

1. Data is loaded by `Load_Dataset` where the FFT and augmentations are created.
2. `data_generator` calls the loader and creates training datasets.
3. The loop in `model_pretrain` loads a batch.
4. The batch is passed through the encoders.
5. Loss is calculated from the representations of the batch.
6. The loss is used for backpropagation.

FFT dimension, spectrum conversion, `fft` vs. `rfft`

- **FFT dimension:** over the time dimension, two sided to keep matching length with the time domain part.
- **Spectrum conversion:** absolute value/magnitude is used.
- `fft` or `rfft`: `fft` is used. This keeps the same length as the original time series.

Active time-domain augmentation; the two frequency operations and how they are combined

- **Time domain:** the paper mentions a bank of multiple different of which a random should be chosen. In practice only jitter is used.
- **Frequency domain:** Frequencies (bins) are added and removed at random. The two augmentation are added which results in the removed frequencies not actually being removed, more halved.

Attributes defining the time encoder, frequency encoder and the two projectors; the four outputs of `TFC.forward` in terms of h and z

- **Time and frequency encoders (same parameters):** length of the time series, dimension of the intermediate layer in the transformer, and number of heads of the attention mechanism.
- **Projectors (same parameters):** Dimension of the first linear layer, dimension of batch normalisation, and dimensions of last linear layer.
- **Relation of outputs to paper notation:**

- `h_time` corresponds to $\mathbf{h}^T$ which is the encoding of the time series by the time encoder, utilizing transformer architecture.
- `z_time` corresponds to $\mathbf{z}^T$ which is the projection of $\mathbf{h}^T$ by the time projector, utilizing linear layers and ReLU.
- `h_freq` corresponds to $\mathbf{h}^F$ which is the encoding of the Fourier transform of the time series by the frequency encoder, utilizing transformer architecture.
- `z_freq` corresponds to $\mathbf{z}^F$ which is the projection of $\mathbf{h}^F$ by the frequency projector, utilizing linear layers and ReLU.

Loss terms in model_pretrain vs. Equations 1–4

- `loss_t = nt_xent_criterion(h_t, h_t_aug)` is the within-time loss from Equation (1).
- `loss_f = nt_xent_criterion(h_f, h_f_aug)` is the within-frequency loss from Equation (2).
- `l_TF = nt_xent_criterion(z_t, z_f)` is the loss between the projection from the frequency- and time-encoders, noted as S_i^{TF} in Equation (3)
- `l_1`, `l_2`, and `l_3` are the negative time-frequency consistency losses, noted as S_i^{pair} in Equation (3).
- `loss = lam*(loss_t + loss_f) + l_TF` is the total loss, which should correspond to Equation (4), but `l_1`, `l_2`, and `l_3` are missing.

Tensors concatenated for the downstream classifier; is the encoder frozen or fine-tuned?

- During fine tuning the representations z_T and z_F are concatenated and used as features for the classifier. The encoder is fine-tuned, and the classifier is trained.
- During testing the representations z_T and z_F are concatenated and used as features for the classifier. Both the encoder and classifier are frozen.

`main.py` → `data_generator` → **Trainer for FD-A** → **FD-B** Number of source and target-training samples actually used, whether `FD_B/val.pt` is loaded, how often the test loader is evaluated, and whether test results influence the reported best result. Discrepancies from the paper’s stated protocol are recorded, not repaired.

- I cannot check the number of samples without loading the files.
- `FD_B/val.pt` is never loaded, only `train.pt` and `test.pt` are loaded.
- The test loader is evaluated once per fine tuning epoch.
- The test influences the best results reported, since the results reported are those which resulted in the best test.
 - Consistency loss is excluded from total loss. This is the paper’s central claim missing.
 - The weight λ of losses varies from training to tuning.
 - They use poly loss instead of pure NT-Xent.
 - Frequency augmentations are summed instead of randomly selected.
 - The paper says that only one frequency bin is augmented, the code augments 10%.

- Only jittering augmentation is used in the time domain. There is no augmentation bank.
- Perturbations of the Fourier transform are not mirrored and the spectrum is therefore not obtainable from a real signal.
- The first set of encoders are specified as 3-layer 1-D ResNets in the paper, while they are 2-layer transformers in the code.
- All datasets are reduced to one channel and length 178.

Device-specific tensor construction in `remove_frequency` and `add_frequency` `torch.cude.FloatTensor` generates a tensor on the GPU. If no GPU is on the machine on which the code is run, the script will crash.

2 Part B: Raw-signal baselines

2.1 Protocol

Datasets: FordA, ArticularyWordRecognition, Epilepsy. Classifiers: logistic regression and RBF-kernel SVM on the vectorised raw signal. Label regimes: 100% and a stratified 10% subset. Three fixed seeds; the 10% subset of a given seed is reused by every method. Hyperparameters selected on training data only.

Fill in.

2.2 Results

Dataset	Model	Labels	Accuracy	Macro-F1
FordA	Logistic regression	100%		
FordA	Logistic regression	10%		
FordA	RBF-SVM	100%		
FordA	RBF-SVM	10%		
AWR	Logistic regression	100%		
AWR	Logistic regression	10%		
AWR	RBF-SVM	100%		
AWR	RBF-SVM	10%		
Epilepsy	Logistic regression	100%		
Epilepsy	Logistic regression	10%		
Epilepsy	RBF-SVM	100%		
Epilepsy	RBF-SVM	10%		

Table 1: Raw-signal baselines. Test accuracy and macro-F1 as mean $\pm$ standard deviation over three paired seeds.

2.3 Comments

Fill in.

3 Part C: Pretrain and evaluate T-Loss and TS2Vec

3.1 Protocol

Per dataset and seed: pretrain on the unlabelled official training series, extract one fixed-length representation per series, freeze the encoder, fit the probe on the 10% and 100% labelled training representations, and evaluate once on the official test set.

Fill in.

Principal representation test The probe treated as the principal representation test is:

Fill in.

3.2 Results

Dataset	Method	Probe	Labels	Accuracy	Macro-F1
FordA	T-Loss	Logistic regression	100%		
FordA	T-Loss	Logistic regression	10%		
FordA	T-Loss	RBF-SVM	100%		
FordA	T-Loss	RBF-SVM	10%		
FordA	TS2Vec	Logistic regression	100%		
FordA	TS2Vec	Logistic regression	10%		
FordA	TS2Vec	RBF-SVM	100%		
FordA	TS2Vec	RBF-SVM	10%		
AWR	T-Loss	Logistic regression	100%		
AWR	T-Loss	Logistic regression	10%		
AWR	T-Loss	RBF-SVM	100%		
AWR	T-Loss	RBF-SVM	10%		
AWR	TS2Vec	Logistic regression	100%		
AWR	TS2Vec	Logistic regression	10%		
AWR	TS2Vec	RBF-SVM	100%		
AWR	TS2Vec	RBF-SVM	10%		
Epilepsy	T-Loss	Logistic regression	100%		
Epilepsy	T-Loss	Logistic regression	10%		
Epilepsy	T-Loss	RBF-SVM	100%		
Epilepsy	T-Loss	RBF-SVM	10%		
Epilepsy	TS2Vec	Logistic regression	100%		
Epilepsy	TS2Vec	Logistic regression	10%		
Epilepsy	TS2Vec	RBF-SVM	100%		
Epilepsy	TS2Vec	RBF-SVM	10%		

Table 2: Frozen-representation classification. Mean $\pm$ standard deviation over three paired seeds.

3.3 Comparison with the raw-signal baselines

Fill in.

Dataset	Method	Pretraining time	Representation dimension
FordA	T-Loss		
FordA	TS2Vec		
AWR	T-Loss		
AWR	TS2Vec		
Epilepsy	T-Loss		
Epilepsy	TS2Vec		

Table 3: Training time and representation dimensionality.

3.4 Training time and representation dimensionality

4 Part D: Robustness to contiguous sensor dropout

4.1 Corruption design

Dropout proportion, number of intervals and replacement convention, fixed before inspecting any result, plus a brief justification. The same corrupted inputs are applied to all methods.

Fill in.

4.2 Results (100%-label models)

Dataset	Method	Accuracy		Macro-F1	
		Clean	Corrupted (Δ)	Clean	Corrupted (Δ)
FordA	Logistic regression (raw)				
FordA	RBF-SVM (raw)				
FordA	T-Loss				
FordA	TS2Vec				
AWR	Logistic regression (raw)				
AWR	RBF-SVM (raw)				
AWR	T-Loss				
AWR	TS2Vec				
Epilepsy	Logistic regression (raw)				
Epilepsy	RBF-SVM (raw)				
Epilepsy	T-Loss				
Epilepsy	TS2Vec				

Table 4: Clean versus contiguous-dropout test performance for the 100%-label models.

4.3 Discussion

Whether any apparent robustness comes from the representation, the classifier or the preprocessing convention.

Fill in.

5 Part E: Focused TS2Vec continuous masking ablation

5.1 Setup

On FordA: standard (binomial) TS2Vec masking versus continuous masking, with encoder, training budget, downstream probe, label regime and the three seeds matched.

Fill in.

5.2 Results

Masking	Accuracy		Macro-F1	
	Clean	Corrupted (Δ)	Clean	Corrupted (Δ)
Binomial (default)				
Continuous				

Table 5: FordA: standard versus continuous TS2Vec masking, clean and contiguous-dropout test sets.

5.3 Interpretation

Does continuous masking change (i) clean performance, (ii) corrupted performance, and (iii) sensitivity to corruption relative to the model’s own clean score?

Fill in.

6 Part F: TF-C for industrial bearing fault classification (FD-A $\rightarrow$ FD-B)

6.1 Execution record

Item	Value
Commands (pretraining)	
Commands (fine-tuning / testing)	
Environment (Python, PyTorch, OS)	
Hardware	
Seed	
Data actually consumed	
Training time	
Checkpoint used	

Table 6: TF-C FD-A $\rightarrow$ FD-B execution record.

6.2 Classification result

Run	Accuracy	Macro-F1
This run (FD-B test)		
TF-C supplement, Table 5		

Table 7: FD-B classification result, compared cautiously with Table 5 of the TF-C supplement. Exact numerical reproduction is not expected.

6.3 What was transferred, where labels entered, and encoder status

What was transferred from FD-A, where FD-B labels entered the pipeline, and whether the final encoder was frozen or updated.

Fill in.

6.4 Material differences between the paper's protocol and the supplied execution path

Subset use, loss composition, validation use and test-set access, based on the Part A inspection.

Fill in.

7 Part G: Synthesis

7.1 When SSL provides a useful representation and when raw nonlinear classification remains competitive

Fill in.

7.2 Whether the effect of SSL becomes clearer with only 10% labels

Fill in.

7.3 Stability of conclusions across the paired seeds

Fill in.

7.4 Role of dataset size, dimensionality and number of classes

Fill in.

7.5 Whether continuous masking improves accuracy, robustness, both or neither

Fill in.

7.6 What the TF-C bearing experiment shows about transfer between operating conditions

Fill in.

7.7 Implications for future multichannel industrial sensor data

Fill in.

8 Experimental rules: compliance notes

- Official test sets preserved as held-out data:

Fill in.

- Test inputs not used for SSL pretraining in the required experiments:

Fill in.

- Normalisation, imputation and feature transforms fitted on training data only:

Fill in.

- 10% label subsets stratified, indices saved:

Fill in.

- Same seeds, label subsets, corruptions and probe settings in paired comparisons:

Fill in.

- Tuning on a training-only validation split or cross-validation:

Fill in.

- Encoder frozen in Parts C–E; Part F follows the supplied TF-C downstream procedure:

Fill in.

- Configurations, software versions, runtime and hardware recorded:

Fill in.

- Implementation departures from the official repositories:

Fill in.

9 Limitations and conclusions

Fill in.

A Optional extension: TF-C on FordA

Fill in.

B Optional extension: FordB noisy-condition experiments

Fill in.

C Per-seed results

Fill in.